

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Системы автоматизированного проектирования»

ОТЧЕТ
по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных
задачах»
на тему «Оценка времени выполнения программ»

Выполнили:

студенты группы 25ВВВ4

Бакин И.В.

Рынденков Д.Ю.

Приняли:

Юрова О.В.

Деев М.В.

Цель работы: изучить как производится оценка выполнения программ.

Ход работы

Задание 1:

1) Вычислить порядок сложности программы (О-символику).

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    clock_t start, end; // объявляем переменные для определения времени выполнения

    int i=0, j=0, r;
    int a[200][200], b[200][200], c[200][200], elem_c;

    srand(time(NULL)); // инициализируем параметры генератора случайных чисел
    while(i<200)
    {
        while(j<200)
        {
            a[i][j]=rand()% 100 + 1; // заполняем массив случайными числами
            j++;
        }
        i++;
    }

    srand(time(NULL)); // инициализируем параметры генератора случайных чисел
    i=0; j=0;
    while(i<200)
    {
        while(j<200)
        {
            b[i][j]=rand()% 100 + 1; // заполняем массив случайными числами
            j++;
        }
        i++;
    }

    for(i=0; i<200; i++)
    {
        for(j=0; j<200; j++)
        {
            elem_c=0;
            for(r=0; r<200; r++)
            {
                elem_c=elem_c+a[i][r]*b[r][j];
                c[i][j]=elem_c;
            }
        }
    }

    return(0);
}
```

$O(A)=O(1)$; $O(B)=O(1)$; $O(C)=O(1)$; $O(D)=O(\max(1, n \cdot 1))=O(n)$;
 $O(E)=O(n \cdot 1)=O(n)$; $O(F)=O(n \cdot 1)=O(n)$; $O(G)=O(n \cdot n)=O(n^2)$; $O(H)=O(1)$;
 $O(I)=O(n \cdot \max(n, 1))=O(n^2)$; $O(J)=O(n \cdot \max(n, 1))=O(n^2)$; $O(K)=O(n \cdot n^2)=O(n^3)$;
 $O(L)=O(\max(1, n^2, 1, n^2, n^3, 1))=O(n^3)$;

2) Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки time.h для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000

При матрицах размером 100 на 100 получается:

t = 0.0000000000000000

При матрицах размером 200 на 200 получается:

```
t = 0.0010000000000000
```

При матрицах размером 400 на 400 получается:

```
t = 0.0090000000000000
```

При матрицах размером 1000 на 1000 получается:

```
t = 0.2170000000000000
```

При матрицах размером 2000 на 2000 получается:

```
t = 2.2290000000000000
```

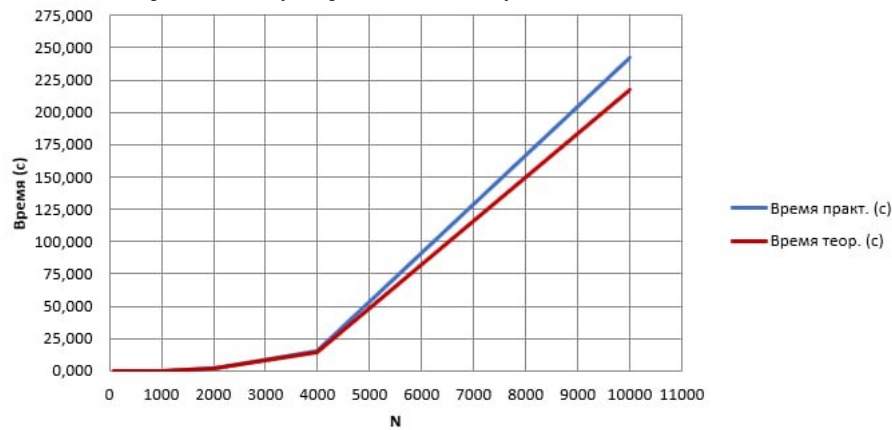
При матрицах размером 4000 на 4000 получается:

```
t = 15.1920000000000000
```

При матрицах размером 10000 на 10000 получается:

```
t = 242.65199999999987
```

3) Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.



Задание 2:

1) Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.

```
Shell: 0.009128  
Quick sort: 0.000857
```

```
Shell: 0.106601  
Quick sort: 0.002033
```

2) Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.

```
Shell: 0.000635  
Quick sort: 0.000903
```

```
Shell: 0.000112  
Quick sort: 0.000142
```

3)Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.

```
Shell: 0.224484
Quick sort: 0.000675
```

```
Shell: 0.012365
Quick sort: 0.000177
```

4)Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.

```
Shell: 0.121023
Quick sort: 0.436505
```

```
Shell: 0.01007
Quick sort: 0.029007
```

5)Оценить время работы стандартной функции qsort, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

```
1: 0.002207
2: 0.000301
3: 0.001138
4: 0.002697
```

```
1: 0.000398
2: 4.7e-05
3: 0.000205
4: 0.000376
```

Вывод: в ходе работы я научился измерять время выполнения программ с помощью библиотеки time.h. Выяснилось, что скорость сортировки зависит от исходных данных, а стандартная qsort работает стабильно быстро на всех наборах.